

ASSIGNMENT SUBMISSION

Name: Devansh Sahu
Program: B.Tech Computer Science and Engineering (AIML)
Institution: LNCT Group of Colleges
Module: React – Conditional Rendering and Lists Lab

Lab Objectives

- Explain various ways of conditional rendering
- Explain how to render multiple components and list components
- Explain keys in React and how to extract components with keys
- Explain React Map, map() function
- Implement conditional rendering in React applications

Prerequisites

- Node.js
- NPM
- Visual Studio Code

Lab Execution Steps

1. Create the React Application

Initialize a new React app named "bloggerapp" using create-react-app.

Screenshot 1: Terminal Output

MINGW64:/c/Users/devansh/projects

```
devansh@LNCT-Desktop MINGW64 ~/projects
```

```
$ npx create-react-app bloggerapp
```

```
Creating a new React app in C:\Users\devansh\projects\bloggerapp.
```

```
Installing packages. This might take a couple of minutes.
```

```
Installing react, react-dom, and react-scripts with cra-template...
```

```
Success! Created bloggerapp at C:\Users\devansh\projects\bloggerapp
```

```
devansh@LNCT-Desktop MINGW64 ~/projects
```

```
$ cd bloggerapp
```

2. Create BookDetails Component (if/else Statement)

Implementing conditional rendering using the traditional if/else approach.

Screenshot 2: VS Code - BookDetails.js

```
src/components/BookDetails.js

import React from 'react';

export default function BookDetails({ title, author, isAvailable }) {
  // Conditional Rendering Method 1: if/else statement
  if (!isAvailable) {
    return (
      <div className="card">
        <h3>{title}</h3>
        <p style={{color: 'red'}}>This book is currently out of stock.</p>
      </div>
    );
  }

  return (
    <div className="card">
      <h3>{title}</h3>
      <p>Author: {author}</p>
      <p style={{color: 'green'}}>Available for checkout!</p>
    </div>
  );
}
```

3. Create BlogDetails Component (Logical && Operator)

Implementing conditional rendering using the logical AND (&&) operator for inline conditionals.

Screenshot 3: VS Code - BlogDetails.js

```
src/components/BlogDetails.js

import React from 'react';

export default function BlogDetails({ title, views, isTrending }) {
  // Conditional Rendering Method 2: Logical && Operator
  return (
    <div className="card">
      <h3>{title}</h3>
      <p>Views: {views}</p>
      {isTrending && <span className="badge trending">🔥 Trending Now!</span>}
    </div>
  );
}
```

4. Create CourseDetails Component (Ternary Operator)

Implementing conditional rendering using the ternary conditional operator (? :).

Screenshot 4: VS Code - CourseDetails.js

```
src/components/CourseDetails.js

import React from 'react';

export default function CourseDetails({ title, instructor, isCompleted }) {
  // Conditional Rendering Method 3: Ternary Operator
  return (
    <div className="card">
      <h3>{title}</h3>
      <p>Instructor: {instructor}</p>
      <div>
        Status: {isCompleted
          ? <span style={{color: 'green'}}>Completed ✓</span>
          : <span style={{color: 'orange'}}>In Progress ⌚</span>
        }
      </div>
    </div>
  );
}
```

5. Integrate Components in App.js (Switch Statement & Map Function)

Using a switch statement for active tabs, generating list components using the `map()` function, and applying unique key attributes.

Screenshot 5: VS Code - App.js

```

import React, { useState } from 'react';
import BookDetails from './components/BookDetails';
import BlogDetails from './components/BlogDetails';
import CourseDetails from './components/CourseDetails';
import './App.css';

export default function App() {
  const [activeTab, setActiveTab] = useState('books');

  // Sample Data Lists
  const books = [
    { id: 1, title: 'React Basics', author: 'John Doe', isAvailable: true },
    { id: 2, title: 'Advanced JS', author: 'Jane Smith', isAvailable: false }
  ];
  const blogs = [
    { id: 1, title: 'Why Hooks?', views: 1500, isTrending: true }
  ];
  const courses = [
    { id: 1, title: 'Fullstack React', instructor: 'Devansh Sahu', isCompleted: false }
  ];

  // Conditional Rendering Method 4: Switch Statement
  const renderContent = () => {
    switch(activeTab) {
      case 'books':
        return books.map(b => <BookDetails key={b.id} {...b} />);
      case 'blogs':
        return blogs.map(b => <BlogDetails key={b.id} {...b} />);
      case 'courses':
        return courses.map(c => <CourseDetails key={c.id} {...c} />);
      default:
        return <p>Select a tab to view details.</p>;
    }
  };

  return (
    <div className="App">
      <h1>Blogger App Dashboard</h1>
      <div className="tabs">
        <button onClick={() => setActiveTab('books')}>Books</button>
        <button onClick={() => setActiveTab('blogs')}>Blogs</button>
        <button onClick={() => setActiveTab('courses')}>Courses</button>
      </div>
      <div className="content-area">
        {renderContent()}
      </div>
    </div>
  );
}

```

6. Start the Application

Run the app to observe the conditional rendering and list map functions locally.

Screenshot 6: Terminal Output

MINGW64:/c/Users/devansh/projects/bloggerapp

```
devansh@LNCT-Desktop MINGW64 ~/projects/bloggerapp
```

```
$ npm start
```

```
Starting the development server...
```

```
Compiled successfully!
```

```
You can now view bloggerapp in the browser.
```

```
Local: http://localhost:3000
```

```
On Your Network: http://192.168.1.100:3000
```

```
Note that the development build is not optimized.
```

```
To create a production build, use npm run build.
```